The Inversion Curse in LLM Fine-Tuning for Paired Code Transformations

SERGE LIONEL NIKIEMA*, University of Luxembourg, Luxembourg
JORDAN SAMHI, University of Luxembourg, Luxembourg
MICHELINE BÉNÉDICTE MOUMOULA, University of Luxembourg, Luxembourg
ALBÉRICK EURASTE DJIRÉ, University of Luxembourg, Luxembourg
ABDOUL KADER KABORÉ, University of Luxembourg, Luxembourg
JACQUES KLEIN, University of Luxembourg, Luxembourg
TEGAWENDÉ F. BISSYANDÉ, University of Luxembourg, Luxembourg

Many software engineering tasks come in pairs, such as translation and back-translation. Fine-tuned code LLMs are usually scored only on the trained direction. We measure both directions with the Bidirectional Reasoning Framework (BRF). We evaluate three paired settings: obfuscation and deobfuscation, translation and back-translation, and compilation and decompilation. Obfuscation is the primary controlled testbed. The inverse spans lexical, structural, and semantic recovery.

We fine-tune six LLMs with forward-only training on three semantics-preserving obfuscation types. We evaluate four reverse prompting strategies. Forward accuracy rises from below 5% to between 12.0% and 81.3% before self-correction, by transformation complexity. Reverse deobfuscation accuracy falls to 0% in every setting we report. Reverse outputs remain executable and pass semantic checks. They stay near-verbatim copies of the obfuscated input. We probe lexical deobfuscation with chain-of-thought prompting. A model can state the correct reverse mapping in its reasoning and still emit forward-style output.

We compare two recovery methods. Bidirectional fine-tuning restores reverse accuracy on lexical and structural tasks. Contrastive fine-tuning restores it only on identifier renaming, from 0% to 43% average across models, without reverse labels. Literal Encryption resists both methods under our setup. Forward-only Python-to-Java fine-tuning lowers Java-to-Python test-pass accuracy from 85.4% to 63.3%. Forward-only C-to-assembly fine-tuning lowers assembly-to-C re-execution from 16.5% to 5.5%. Under BRF, forward success does not preserve the inverse capability assumed by paired SE workflows.

Additional Key Words and Phrases: Inversion Curse, Directional Capability Loss, Bidirectional Reasoning, Fine-Tuning, LLMs

1 INTRODUCTION

Many software engineering tasks come in pairs. Obfuscation and deobfuscation

This paper asks whether forward fine-tuning preserves the reverse capability of LLMs. We ground the question in three paired settings: ① obfuscation and deobfuscation, ② translation and back-translation, and ③ compilation and decompilation. Obfuscation provides the primary controlled testbed. Its forward direction is cheap to supervise with existing tools

The finding is negative, and ordinary forward evaluation would not expose it. Across six LLMs, three semantics-preserving obfuscation types, and four prompting strategies, standard fine-tuning raises forward accuracy to between

*Corresponding author (lionel.nikiema@uni.lu)

Authors' Contact Information: Serge Lionel Nikiema, University of Luxembourg, Luxembourg, Luxembourg, ; Jordan SAMHI, University of Luxembourg, Luxembourg, Luxembourg, jordan.samhi@uni.lu; Micheline Bénédicte Moumoula, University of Luxembourg , Luxembourg, Luxembourg, micheline.moumoula@uni.lu; Albérick Euraste Djiré, University of Luxembourg, Luxembourg, Luxembourg, euraste.djire@uni.lu; Abdoul Kader Kaboré, University of Luxembourg, Luxembourg, Luxembourg, abdoulkader.kabore@uni.lu; Jacques Klein, University of Luxembourg, Luxembourg, Luxembourg, jacques.klein@uni.lu; Tegawendé F. Bissyandé, University of Luxembourg, Luxembourg, Luxembourg, tegawende.bissyande@uni.lu.

2026. Manuscript submitted to ACM

12.0% and 81.3% before self-correction, following a complexity ordering from lexical to semantic transformations, while reducing reverse (i.e., deobfuscation) accuracy to 0%. The reverse failure takes a specific form. The fine-tuned models return semantically valid programs that stay near-verbatim copies of the obfuscated input, so they remain at the transformed endpoint rather than moving back toward a readable equivalent. The two additional settings reproduce the same direction of effect. Forward-only fine-tuning on Python-to-Java lowers Java-to-Python performance from 85.4% to 63.3%, and forward-only fine-tuning on C-to-assembly lowers assembly-to-C performance from 16.5% to 5.5%.

The Bidirectional Reasoning Framework (BRF) makes the failure visible. It evaluates a fine-tuned model on both the trained transformation and its inverse, then reports the gap between the two directions. The framework does not require recovery of the exact original string. It asks a more basic question. Once a model has been adapted to produce a forward transformation, does it still move code in the reverse direction at all? The behavior we observe is systematic. A fine-tuned model generates almost exclusively in the trained direction even when the reverse prompt is explicit and task-relevant

This behavior echoes the reversal curse, which reports that an autoregressive LLM trained on a fact stated as “A is B” often fails to answer the reversed query “B is A”

Moving the question into software engineering changes what can be asked and what is at stake. A program transformation and its inverse are executable procedures with preconditions, search spaces, and correctness criteria, so the inverse is no longer a single reversed fact but a task that can be validated by compilation and testing. The inverse is also graded. Our transformations run from a near-lexical identifier mapping to structural pruning and to algorithmic recovery of encrypted literals, which lets us study how the asymmetry scales with the difficulty of the inverse rather than whether it merely appears. Because the inverse is executable and graded, software engineering offers a setting where the phenomenon can be measured, decomposed, and connected to deployment.

That setting also raises the stakes. In a deployed pipeline, a missing inverse transformation can silently disable deobfuscation, decompilation, rollback, or back-translation, even when the trained direction continues to pass its own tests. A concrete case is the organization that deploys a single fine-tuned code model as a shared assistant. When that model is adapted for one dominant task, such as a large translation or hardening effort, the same parameters serve every developer who later relies on the inverse direction, and the inverse is rarely re-validated after the adaptation. The governing question for our community is therefore whether post-training adaptation preserves the adjacent capabilities that real workflows assume remain available. We answer it with direct measurement on six models and three transformation families, and we trace the asymmetry to ordinary downstream fine-tuning rather than to pretraining data order.

We address three research questions. First, does forward fine-tuning preserve or improve reverse capability in the absence of reverse supervision? Second, what evidence separates an ordinarily hard inverse task from a direction-specific failure induced by fine-tuning? Third, can bidirectional supervision or contrastive regularization recover reverse capability, and what does the recovery pattern reveal about the underlying mechanism? The answers hold across model families. Standard fine-tuning fails to preserve reverse capability. Balanced bidirectional supervision recovers the lexical and structural inverses but not the semantic inverse. Contrastive fine-tuning recovers reverse capability only on the lexical transformation.

The chain-of-thought probe sharpens the mechanism. On identifier renaming, a fine-tuned model states the correct reverse mapping in its reasoning trace and then emits the obfuscated code unchanged. We do not read the trace as a faithful report of internal state. We read it as empirical evidence that the reverse mapping can be surfaced at inference

time without being applied in the final generation. The gap fits a generation-time directional bias acting on top of any representational effect.

We make the following contributions:

① We identify the inversion curse, a directional capability loss in which forward fine-tuning of a code LLM improves the trained direction while driving reverse capability to zero.
② We define BRF, a low-cost diagnostic that exposes the asymmetry by evaluating a fine-tuned model in both directions.
③ We give empirical evidence that the lexical case reflects a reasoning-generation gap rather than absence of the reverse mapping.
④ We derive implications for validating, deploying, and repairing fine-tuned code LLMs, and we corroborate the obfuscation findings on translation and compilation.

2 BACKGROUND

2.1 Paired Program Transformations

Many software engineering tasks come in pairs. Translation has back-translation. Compilation has decompilation. Obfuscation has deobfuscation. Research on bidirectional transformations studies such pairs in a formal way

Learned code transformations rarely meet this condition. Identifier renaming discards the original names. Deobfuscation tools can only predict plausible names

Prior work gives no definition of an inverse pair for learned code transformations. We use the following one. Let $\mathcal{P}$ be a set of programs. Let $p \equiv q$ denote that p and q pass the same test suite. Let d be a distance over program form, such as lexical or structural similarity.

Definition 1 (Forward/inverse pair). *A transformation $T : \mathcal{P} \to \mathcal{P}$ is a valid forward task if $T(p) \equiv p$ for every $p \in \mathcal{P}$. The inverse task of T takes $T(p)$ as input. A solution is any program q with $q \equiv T(p)$ and $d(q, p) < d(T(p), p)$.*

The definition keeps semantic equivalence. It replaces exact recovery with measurable progress toward the source form. It accepts many correct inverse outputs. Obfuscation, translation, and compilation satisfy it. Mutation and repair do not. A mutant changes behavior. BRF (Section 3) applies to the pairs that satisfy Definition 1.

2.2 Obfuscation as a Controlled Testbed

Obfuscation turns a program into an equivalent program that is harder to understand

Deobfuscation has a long research history. Yadegari et al. simplify obfuscated executables through semantics-based analysis

This history explains our choice of testbed. The forward direction is cheap to supply with existing obfuscators. Forward training pairs are therefore abundant. The inverse direction must recover lost information. Reverse supervision is therefore scarce. The three types range from lexical to semantic changes. This range lets us relate the size of the effect to the difficulty of the inverse. The translation and compilation checks extend the study to two other pairs (Section 5.5).

2.3 Adapting Code LLMs to One Task

LLMs now support a wide range of software engineering tasks

Adapted models are then shared and reused. Jiang et al. study pre-trained model reuse on Hugging Face

2.4 Motivation

Prior work offers three relevant findings. Autoregressive models can learn a relation in one direction only

These findings come from separate settings. The reversal curse is studied on factual recall. Forgetting is studied on tasks unrelated to the new one. Round-trip evaluation is applied to a model that has not been adapted to one direction. Studies of LLMs on translation, decompilation, and obfuscation evaluate the direction they target

This question has practical weight. Adapted checkpoints are shared and reused with incomplete documentation

Our study complements these lines of work. It combines existing ideas in one protocol. It borrows the two-direction view from round-trip evaluation. It borrows a control on other tasks from forgetting studies. It adapts reverse training and contrastive learning as recovery probes

3 DIRECTIONAL CAPABILITY LOSS AND BRF

The failure mode we study is not ordinary underperformance on a difficult reverse task. A hard inverse can arise because the reverse mapping is underspecified, structurally complex, or poorly represented in the training data. Directional capability loss is a stronger condition. It arises when forward fine-tuning causes the model to generate in the trained direction even when the prompt requests the inverse. The distinction has practical weight in software engineering, where forward and reverse tasks are operationally adjacent. A model that is competent only in the trained direction can still be unsafe inside a workflow that depends on rollback, inspection, or reverse analysis.

Let T be a semantics-preserving program transformation that maps an original program C_{orig} to a transformed program C_{trans}, and let M_T be a model fine-tuned only on the forward direction of T. BRF evaluates M_T on the forward mapping $C_{\text{orig}} \rightarrow C_{\text{trans}}$ and on the reverse mapping $C_{\text{trans}} \rightarrow C_{\text{pred}}$. Reverse success cannot be defined by execution alone. Since C_{trans} is already semantically equivalent to C_{orig}, a model that copies the transformed input back will still compile and pass the tests. Reverse evaluation must therefore ask whether the output moves toward a readable program equivalent to the original, rather than remaining at the transformed endpoint.

We measure that asymmetry with a bidirectional gap. For a given model and transformation, BRF computes forward accuracy $\text{Acc}_{\text{fwd}}(M_T)$ and reverse recovery $\text{Acc}_{\text{rev}}(M_T)$ and defines:

$$\Delta_{\text{BRF}}(M_T) = \text{Acc}_{\text{fwd}}(M_T) - \text{Acc}_{\text{rev}}(M_T). \tag{1}$$

A large gap does not, on its own, diagnose the phenomenon, because some inverse tasks are harder than their forward counterparts. We classify the pattern as the inversion curse only when three conditions hold jointly: ❶ Reverse accuracy is near zero, ❷ the outputs remain measurably closer to the transformed input than to the original program, and ❸ stronger prompting does not improve the reverse result. Under these conditions the model behaves as though adaptation has bound its generation policy to the trained direction.

BRF is deliberately lightweight. It needs no second model, no architectural change, and no reverse-direction training data. It adds a single evaluation pass and surfaces a capability that forward validation leaves untested. The remainder of the paper instantiates BRF on obfuscation and deobfuscation. The diagnostic applies to any paired software engineering task that satisfies Definition 1, including translation and back-translation and compilation and decompilation.

4 CONTROLLED TESTBED AND EXPERIMENTAL DESIGN

We study directional capability loss across three software engineering settings. The main experiments use obfuscation and deobfuscation, which give controlled access to forward pairs, executable validation, and inverse tasks of graded

structural complexity. We then evaluate whether the same phenomenon generalizes to two additional settings: Python-to-Java translation with Java-to-Python reversal, and C-to-assembly compilation with assembly-to-C reversal.

4.1 Models

We evaluate the six LLMs in Table 1. Four are open-source 7B-class models that support LoRA fine-tuning on academic hardware, and two are proprietary instruction-tuned models that exposed a fine-tuning interface at the time of the experiments. The set is not a leaderboard. It is chosen so that the phenomenon can be tested across open and proprietary systems, general and code-specialized pretraining, and reasoning-distilled and standard decoder-only models. More recent and more capable models exist, but limited fine-tuning access and their size prevented us from using them; Section **??** discusses this limitation.

Table 1. Evaluated LLMs.

Model	Origin	Specialization	Fine-Tuning
Qwen2.5-7B	Open-source	General-purpose	LoRA
Qwen2.5-Coder-7B	Open-source	Code-specialized	LoRA
DeepSeek-R1-Distill-Qwen-7B	Open-source	Reasoning-distilled	LoRA
Mistral-7B	Open-source	General-purpose	LoRA
GPT-3.5-Turbo	Proprietary	Instruction-tuned	API
GPT-4.1-mini	Proprietary	Instruction-tuned	API

4.2 Obfuscation Tasks

The forward direction maps C_{orig} to an obfuscated program C_{trans}. The reverse direction asks whether a fine-tuned model can map C_{trans} back to a semantically equivalent program that is closer to the original readable code. We implement three semantics-preserving transformations with the open-source *Obfuscator* tool

The ordering from Identifier Renaming through Dead Code Insertion to Literal Encryption underpins our interpretation. Identifier Renaming admits many acceptable reversals, yet its inverse stays local and syntactic. Dead Code Insertion forces the model to decide which inserted statements are semantically inert and removable. Literal Encryption forces recovery of hidden values by reasoning over the inserted decryption logic. The three tasks therefore span lexical, structural, and semantic inverses while the forward tool is held fixed.

4.3 Additional Settings

Two further checks operate outside obfuscation. For translation, we fine-tune GPT-4.1-mini on 1,000 Python-to-Java pairs drawn from matched CodeNet Python800/Java250 problems and evaluate the untrained reverse direction on 300 ConDefects programs

4.4 Datasets

We need executable programs with reliable test suites because semantic equivalence is the only defensible ground truth for both obfuscation and deobfuscation. We therefore use CodeNet

(a) Identifier Renaming

Original

```java
int calculateSum(
      int[]
      numbers) {
   int total =
      0;
   for (int num
      : numbers)
       {
       total +=
      num;
   }
   return total;
}
```

Obfuscated

```java
int lIii(int[]
      Iiii) {
   int liii = 0;
   for (int iIIi
       : Iiii) {
       liii +=
      iIIi;
   }
   return liii;
}
```

(b) Dead Code Insertion

Original

```java
int getMax(int a,
      int b) {
   if (a > b) {
       return a;
   }
   return b;
}
```

Obfuscated

```java
int getMax(int a,
      int b) {
   if (false) {
       System.out.
      println("
      dead");}
   if (a > b) {
       return a;}
   return b;
}
```

(c) Literal Encryption

Original

```java
void greet() {
   String msg =
      "Hello!";
   System.out.
      println(
      msg);
}
```

Obfuscated

```java
void greet() {
   String msg =
      decrypt("
      Xy9K...");
   System.out.
      println(
      msg);
}
```

Fig. 1. Code transformation examples. Original and obfuscated snippets for Identifier Renaming, Dead Code Insertion, and Literal Encryption.

The split between training and evaluation data is deliberate. CodeNet supplies breadth for fine-tuning, and ConDefects supplies held-out programs for validation. The design lets us ask two questions at once, whether a model learns a forward transformation that transfers to new programs, and whether the same adaptation preserves any inverse capability on unseen code. The first answer is frequently affirmative. The second answer is the central finding of the paper.

4.5 Fine-Tuning Protocol

For the open-source models we use LoRA

We calibrated the training size and schedule in a preliminary study. For the open-source models, we compared 5,000, 10,000, and 20,000 forward pairs per transformation, and forward performance was highest with 10,000 pairs. Training loss and held-out forward accuracy saturated by the third epoch, so we train for three epochs. For the proprietary API models, we run instruction fine-tuning through the provider interface

We compare three training strategies. Standard fine-tuning (SFT) trains on the forward mapping from original to obfuscated code alone. Balanced bidirectional fine-tuning (BFT) adds explicit supervision in both directions and so establishes a recovery ceiling for the reverse task. Contrastive fine-tuning (CFT) keeps the forward generation objective and adds semantic triplets $(C_{\text{anchor}}, C_{\text{positive}}, C_{\text{negative}})$, where the positive is an obfuscated version of the anchor and the negative is an unrelated program. CFT is not designed to maximize forward accuracy. It tests whether preserving semantic proximity in representation space reduces the reverse failure without any reverse labels.

Table 2. Hyperparameter configuration for all models and training strategies.

Model	LoRA r	LoRA α	LR	Batch	Epochs	Temp.
Qwen2.5-7B	16	32	2e-4	4	3	0
Qwen2.5-Coder-7B	16	32	2e-4	4	3	0
DeepSeek-R1-Distill-Qwen-7B	16	32	2e-4	4	3	0
Mistral-7B	16	32	2e-4	4	3	0
GPT-3.5-Turbo	—	—	auto	4	3	0
GPT-4.1-mini	—	—	auto	4	3	0

4.6 Hyperparameters

Table 2 lists the main hyperparameters. Open-source models use LoRA rank 16 and $\alpha = 32$, a configuration standard for 7B-scale adaptation

4.7 Evaluation Metrics

No single metric shows whether code has been obfuscated or deobfuscated. We therefore combine a semantic check with several complementary syntactic measures.

- **Semantic correctness.** An output is valid if it ① compiles, ② runs without runtime errors, and ③ passes all test cases of the original program. This check applies to both directions and confirms that the transformation preserves behavior.
- **Syntactic similarity (S).** CodeBLEU
- **Readability (R).** The model of Scalabrino et al.
- **AST-based measures.** We parse programs with tree-sitter
- **Complexity.** We also report token count, computed with a BERT-base tokenizer

Reverse recovery decision rule. Semantic validity alone is not enough. An unchanged obfuscated input also passes the same tests. Reverse success requires the semantic check plus joint movement toward the original on all three channels: CodeBLEU, identifiers, and structure. We require $S(C_{\text{deobf}}, C_{\text{orig}}) > S(C_{\text{tool}}, C_{\text{orig}})$, $AST_{\text{synt}}(C_{\text{deobf}}, C_{\text{orig}}) < AST_{\text{synt}}(C_{\text{tool}}, C_{\text{orig}})$, and $AST_{\text{struct}}(C_{\text{deobf}}, C_{\text{orig}}) > AST_{\text{struct}}(C_{\text{tool}}, C_{\text{orig}})$. There is no similarity threshold. We compare the model output to the tool output on the same original program. All three comparisons must improve at once.

5 THE INVERSION CURSE

The argument proceeds in four steps. We first evaluate what the unfine-tuned models can do on the obfuscation tasks, since a post-training reverse failure is stronger evidence of directional loss when the relevant capability existed beforehand. We then measure how standard forward fine-tuning affects the trained direction. We apply BRF to the reverse direction, where the same models that learn the forward transformation fail to move code back toward the original. We close with the translation and compilation checks, which reproduce the directional trend outside the primary testbed.

5.1 Baseline Behavior Before Fine-Tuning

We prompt each unfine-tuned model with a simple zero-shot instruction to obfuscate the input program under each of the three transformation types: Identifier Renaming, Dead Code Insertion, and Literal Encryption. We evaluate the outputs semantically with compilation, execution, and test pass. We evaluate them syntactically with AST similarity, CodeBLEU,

readability, token count, and cyclomatic complexity. The open-source models produce executable outputs but apply almost no transformation. Figure 2(a) shows low AST_{synt} change. Figure 2(b) shows AST_{struct} near one. Figure 2(c)–(d) show CodeBLEU and readability close to the original program. Their token counts move by under 5%. Their CodeBLEU, readability, and cyclomatic complexity stay close to the original program.

To check whether this reflects the wording of the task, we also described each transformation explicitly without using the term "obfuscation". We asked the models to rename variables, to insert non-functional code blocks that do not change program behavior, and to replace literals with encrypted forms. Under these instructions, the open-source models renamed between 23% and 38% of unique identifiers, but made no change for dead code insertion or literal encryption. One possible explanation is that these models rarely saw obfuscation during training and therefore do not recognize it as a task. They behave as conservative code generators, not as obfuscators. This baseline matters for interpretation. A later reverse failure is unlikely to stem from the model having already mastered a demanding forward transformation, because the forward transformation is barely present at baseline.

The GPT models show a different baseline pattern. Figure 2(a) shows moderate lexical change on Identifier Renaming. They replace roughly 69% of identifiers ($AST_{\text{synt}} = 0.69$), but they produce much less lexical change on Dead Code Insertion and Literal Encryption, where AST_{synt} is 0.185 and 0.315. Figure 2(b) shows that structural similarity remains high across the board, with AST_{struct} between 0.78 and 0.99. The baseline changes are mostly lexical rather than structural. About 37% of GPT outputs fail because of compilation errors, runtime exceptions, or failed tests. This suggests that attempts to modify the code introduce semantic alterations that break executability or test correctness.

A manual review of 100 baseline outputs, split evenly between passing and failing cases, explains these aggregate scores. Identifier substitutions are generic names such as `x` and `y` rather than deliberate obfuscation. The code structure stays intact even under prompt-driven obfuscation. Literal Encryption degrades into dead-code additions rather than genuine replacement of literal values. The baseline behavior is shallow. The models imitate the surface form of a transformation without producing the transformation itself.

The rule-based obfuscator provides a reference point rather than a learning baseline. It passes tests in 100% of cases and produces large changes on every metric. Figure 2(c) shows CodeBLEU relative to the original dropping to 0.50 on Identifier Renaming, 0.31 on Dead Code Insertion, and 0.22 on Literal Encryption. These three values are means over the full evaluation set. Figure 2(d) shows readability falling from 0.79 to as low as 0.19. Token count rises by 377%, and cyclomatic complexity rises by 45%. Overall, Figure 2 shows how far the baseline LLM outputs remain from that transformation profile.

Baseline reverse behavior. We measure the reverse direction before any fine-tuning, so that the post-fine-tuning reverse results have a point of comparison. The results indicate that they recover a measurable but partial fraction of the original program on all three transformations. For GPT-4.1-mini, Acc_{rev} is 48.0% on Identifier Renaming, 49.0% on Dead Code Insertion, and 32.4% on Literal Encryption. GPT-3.5-Turbo reaches 40.2%, 44.6%, and 16.8%. Dead Code Insertion sits slightly above Identifier Renaming for both API models. Literal Encryption remains lowest. As in the forward baseline, many unsuccessful cases fail at compilation, runtime, or test pass. The recovery is moderate, not complete. It registers as movement toward the original on the syntactic metrics rather than as a return to the original program. For GPT-4.1-mini, mean CodeBLEU against the original rises from $S(C_{\text{tool}}, C_{\text{orig}}) = 0.367$ to $S(C_{\text{deobf}}, C_{\text{orig}}) = 0.440$ on Identifier Renaming, from 0.254 to 0.426 on Dead Code Insertion, and from 0.190 to 0.342 on Literal Encryption. Identifier-set overlap with the original rises. Under our divergence definition, that corresponds to AST_{synt} drops of 0.289, 0.214, and 0.194 relative to the tool-obfuscated baseline. The outputs sit between the tool-obfuscated input and the original code. They are closer to the original but short of full restoration.

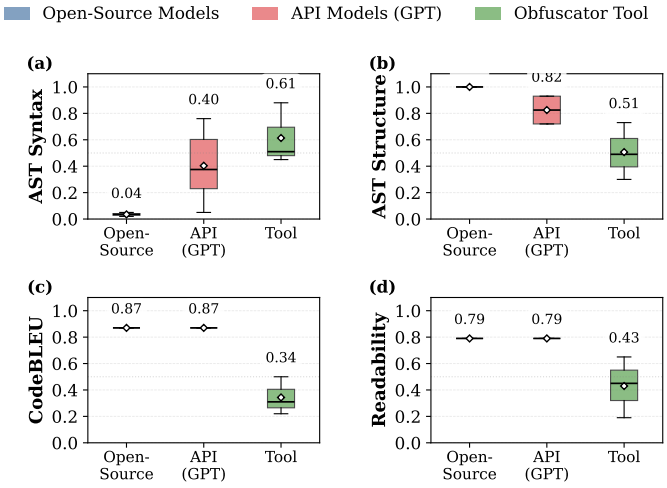


Fig. 2. Distribution of baseline performance across LLMs.

The open-source models, consistent with their near-zero forward transformation at baseline, show negligible reverse recovery before fine-tuning as well. These results suggest that: these models may lack the instruction-following capacity to interpret the reverse prompt without task-specific context or the reverse transformation may exceed their zero-shot reasoning ability; or the scarcity of explicit deobfuscation examples in their pretraining data may make the mapping unrecognizable. Distinguishing these factors is left for future work.

5.2 Forward Performance After Standard Fine-Tuning

We then fine-tune each model with standard fine-tuning (SFT) on forward pairs only. Each training example maps an original Java program to its tool-obfuscated counterpart under one transformation type. The instruction used during fine-tuning is as follows:

Fine-Tuning Prompt Template

`Obfuscate the following Java code by <Obfuscation Technique> while preserving its functionality:`
Input: `<Original code>`
`Here's the obfuscated version preserving functionality:`
Output: `<Obfuscated code>`

Here `<Obfuscation Technique>` is Identifier Renaming, Dead Code Insertion, or Literal Encryption. `<Original code>` is the input and `<Obfuscated code>` is the expected tool output. After fine-tuning, we evaluate on the held-out ConDefects programs with the same instruction format. Figure 4 and Figure 5 show that standard fine-tuning raises forward performance on all three transformations, with gains that vary by task type.

The forward results follow a stable complexity ordering. Identifier Renaming is learned best, at 68.9% average success and 81.3% for GPT-4.1-mini. Dead Code Insertion is harder, at 53.1% average success and 72.7% for Mistral. Literal Encryption is hardest, at 20.0% average success and 29.7% for Mistral. The ordering holds across models, which indicates that fine-tuning is most effective when the target transformation reduces to local substitution and least effective when it requires algorithmic manipulation of program values.

We then apply an iterative auto-correction loop to failed forward outputs. When a generation fails to compile, execute, or pass tests, we feed the error message back to the model with a correction prompt. We report results after five iterations. Further iterations yield no additional gain under our setup. The correction prompt is as follows:

Self-Correction Prompt

System: You are a code obfuscator. The previous obfuscated code did not work. Here is the code and the error. Please correct the code so it compiles and passes all tests.

User:

Original code (for reference):

```
<Original Code>
```

Previous obfuscated code:

```
<Generated Code>
```

Error message:

```
<Compiler/Runtime Error or Failed Test Case>
```

Please correct the code.

Auto-correction raises forward pass rates. The average gain is 16.6% on Identifier Renaming, 19.8% on Dead Code Insertion, and 13.8% on Literal Encryption. GPT-3.5-Turbo rises from 76.3% to 96.0% on Identifier Renaming. GPT-4.1-mini rises from 60.0% to 88.7% on Dead Code Insertion. GPT-3.5-Turbo rises from 22.3% to 46.7% on Literal Encryption. GPT models gain more than the open-source models on average (20.8 percentage points versus 14.7 across the three transformations). The gap is concentrated on Dead Code Insertion and Literal Encryption. Figure 4 reports the corrected pass rates.

Average gains stay high on Identifier Renaming and Dead Code Insertion, where many failures are still recoverable from compiler or test feedback. Gains shrink on Literal Encryption, where the failure is usually a wrong program behavior rather than a fixable syntax fault. Under our setup, self-correction therefore repairs many surface errors. It does not close the gap on Literal Encryption. The task ordering after correction matches the ordering before correction.

We break down the 2,705 failed forward samples after SFT and before self-correction, out of 5,400 evaluations.[1] Of these failures, 1,667 (62%) compile and run but fail tests, 926 (34%) crash at runtime, and 112 (4%) fail to compile. Figure 3 shows the distribution across models and transformations. The pattern varies by task. On Identifier Renaming, 73% of failures are incorrect outputs and 21% are runtime crashes. On Dead Code Insertion, incorrect outputs remain dominant at 69%. On Literal Encryption, runtime crashes rise to 45% and incorrect outputs fall to 52%. Compilation failures stay low on every task (about 3%–6% of failures per transformation). Models often emit code that looks well-formed, while they still miss the required functional behavior. Appendix C.2 reports the full breakdown.

The similarity metrics show that forward fine-tuning shifts model behavior in a targeted way. On Identifier Renaming, $S(C_{\text{obf}}, C_{\text{orig}})$ stays between 0.42 and 0.50 while $S(C_{\text{obf}}, C_{\text{tool}})$ stays between 0.49 and 0.51, a moderate lexical shift toward the tool pattern. On Dead Code Insertion and Literal Encryption, $S(C_{\text{obf}}, C_{\text{orig}})$ falls to between 0.12 and 0.27 while $S(C_{\text{obf}}, C_{\text{tool}})$ rises to between 0.59 and 0.73. Readability follows, from 0.79 on the original code to 0.57 on the fine-tuned outputs. Standard fine-tuning thus teaches the forward surface pattern even where full semantic fidelity is not reached.

[1] Six models, three transformations, 300 evaluation programs each.

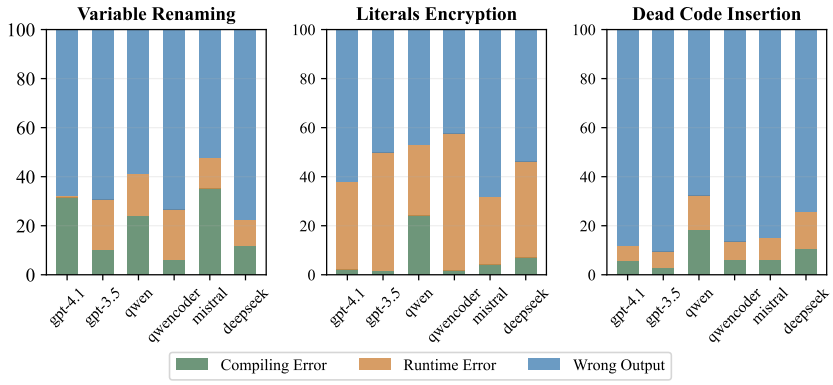


Fig. 3. Error-type distribution for failed forward outputs after SFT. Compilation failures are rare. Runtime crashes and incorrect outputs that still compile account for most failures.

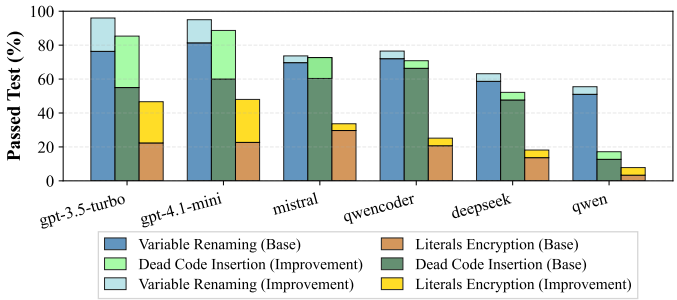


Fig. 4. Fine-tuned LLM performance on forward obfuscation.

5.3 Reverse Evaluation with BRF

We prompt every fine-tuned model to deobfuscate tool-generated obfuscated code under four strategies, namely direct instructions, few-shot prompting, chain-of-thought prompting, and few-shot prompting combined with chain-of-thought. Appendix B lists the full prompts. If forward fine-tuning had preserved any usable inverse capability, at least one of these strategies would yield outputs that move toward the original program. None of them does.

Across all six models, all three transformations, and all four prompting strategies, no output shows net positive movement on any evaluation channel toward the original program. Reverse recovery is 0% under the Acc_{rev} rule. Against the reverse baseline of Section 5.1, the unfine-tuned API models succeed on 17% to 49% of evaluation programs under the same protocol. This is a drop to zero. The moderate deobfuscation present before fine-tuning disappears after forward-only fine-tuning.

This zero is also not an artifact of the word “deobfuscation”. A model might fail a reverse prompt because it does not recognize the task label, even if some inverse capability remained. As in the baseline wording check, we therefore also prompt the fine-tuned models with reverse instructions that never use that term. We ask them to restore meaningful identifier names, to remove non-functional dead-code blocks, and to decrypt AES-encrypted literals back to their original values. Under these formulations, the models still emit code that is essentially identical to the obfuscated input. No evaluation channel moves toward the original program. The result holds even for Identifier Renaming, where reverse

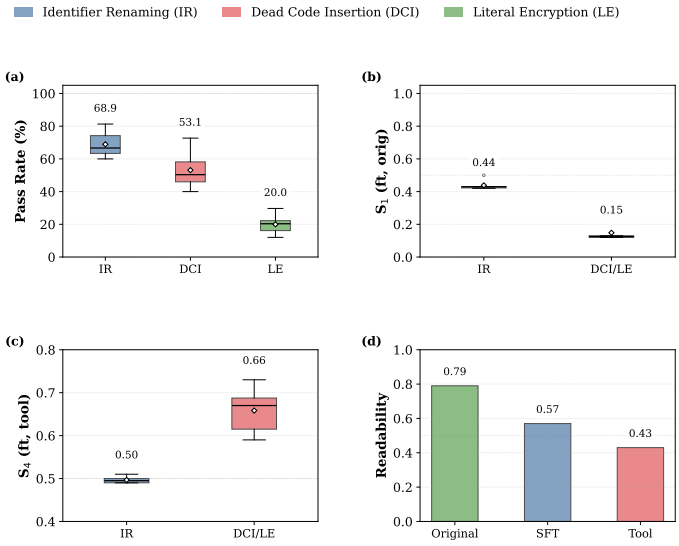


Fig. 5. Forward performance after standard fine-tuning. Pass rates and similarity trends show that models learn the forward syntactic pattern more reliably than they preserve semantic correctness on the harder tasks.

recovery was measurable before fine-tuning (up to 48.0% for GPT-4.1-mini). Prompt reformulation does not restore the inverse mapping. The full prompts appear in Appendix B.

The zero is not a collapse of generation. The reverse outputs pass the semantic validity gate. They compile and preserve the behavior of the obfuscated input. They fail because they do not move toward the original code. On Identifier Renaming, AST_{synt} stays near 0.01 against the obfuscated input, so identifiers barely change. AST_{struct} stays at 1.0, so the control-flow skeleton is untouched (Figure 6(b)). Token counts move within 3% of the obfuscated input. These channels support near-verbatim reproduction of the obfuscated program. CodeBLEU against the obfuscated input is lower, with $S(C_{\text{deobf}}, C_{\text{tool}})$ between 0.65 and 0.79, while $S(C_{\text{deobf}}, C_{\text{orig}})$ is 0.40. That gap is consistent with formatting and local edits that do not restore the original program. Dead Code Insertion and Literal Encryption repeat this pattern. Figure 6(a) shows the concentration near the obfuscated endpoint across all three transformations. Readability does not recover. On Identifier Renaming, $R(C_{\text{deobf}})$ stays between 0.55 and 0.69, below the original readability of 0.79. Dead Code Insertion and Literal Encryption show no readability gain (Figure 6(c)).

Prompting does not change the outcome. Few-shot examples and chain-of-thought reasoning help when a model holds the underlying capability and needs better task framing. Here they leave the direction of generation untouched. Figure 6(d) shows that similarity to the obfuscated input stays high under Simple, Few-Shot, CoT, and augmented prompts. The models emit the obfuscated form even when the prompt requests the reverse, and even when the reasoning trace has begun to describe the reverse mapping (Figure 7). This persistence across prompt forms separates a prompt mismatch from directional capability loss.

> ☞ **Finding.** Standard fine-tuning creates a large bidirectional gap. The same models that reach 12.0–81.3% forward accuracy before self-correction produce 0% successful deobfuscation. Identifier, structure, and size metrics show near-verbatim reproductions of the obfuscated input. CodeBLEU stays closer to the obfuscated endpoint than to the original, without reaching full token identity. This is the defining behavioral signature of the inversion curse.

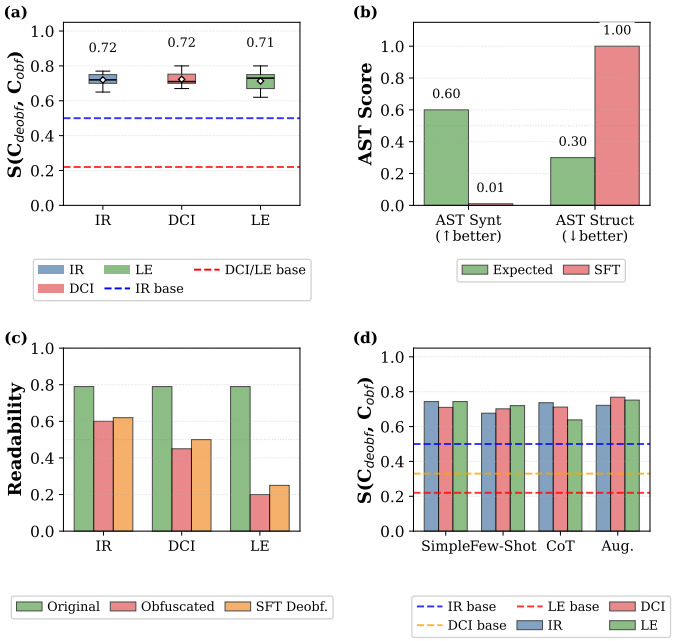


Fig. 6. Reverse performance after standard fine-tuning. Across models and task types, outputs remain concentrated near the obfuscated endpoint instead of moving back toward the original program.

Table 3. Directional loss versus global forgetting. Forward-obfuscation fine-tuning collapses the trained inverse while held-out capabilities stay near baseline. Values are percentages; Δ in percentage points.

Task	Baseline	Post-FT	Δ
Reverse deobfuscation (Java)	48.0	0.0	−48.0
Python→C (re-exec.)	26.8	22.0	−4.8
Code QA (MBPP)	6.0	5.7	−0.3

5.4 Directional Loss, Not Global Forgetting

A large forward-to-reverse gap admits two readings. Adaptation may have suppressed the specific inverse of the trained transformation, or it may have degraded the model broadly, in which case unrelated capabilities would fall alongside the inverse. We separate the two by evaluating the forward-obfuscation checkpoint, with no further fine-tuning, on its trained inverse and on two tasks absent from the obfuscation corpus: Python-to-C translation scored by re-execution, and MBPP code generation. All conditions use identical prompts, decoding, and scoring.

The inverse collapses while the other tasks do not. Reverse deobfuscation falls from 48.0% to 0.0% (−48 pp), whereas Python-to-C moves by −4.8 pp and MBPP by −0.3 pp. Global forgetting would pull those two tasks down together with the inverse. Instead the drop is confined to the direction paired with the trained transformation. The failure is directional, which is what separates the inversion curse from catastrophic forgetting and licenses the name.

5.5 Translation and Compilation Checks

The two additional settings reproduce the directional trend under a lighter protocol. For translation, forward-only fine-tuning on Python-to-Java lowers Java-to-Python reverse accuracy from a baseline of 85.4% to 63.3%. For compilation, forward-only fine-tuning on C-to-assembly lowers assembly-to-C reverse accuracy from 16.5% to 5.5%. Allowing up to three generations raises translation reverse accuracy to 82.3% (below the baseline pass@1), while the same allowance barely moves compilation reverse accuracy, at 8.5%. The pass@3 gap between the two settings distinguishes degraded precision from directional lock-in: the Java-to-Python capability is weakened but accessible under repeated sampling, whereas assembly-to-C recovery is blocked regardless of the number of attempts.

These checks carry less mechanistic detail than the obfuscation study, yet they establish that the directional pattern is not confined to deobfuscation. The milder loss under translation and the severe loss under compilation also track the complexity ordering of the controlled testbed. Reverse performance degrades less when the inverse approximates a constrained mapping between two high-level languages, and more when it requires reconstruction from a lower-level representation.

6 MECHANISM AND RECOVERY UNDER BIDIRECTIONAL SUPERVISION

The previous section established what standard fine-tuning does. This section asks why the reverse failure takes the form it does, and how much of it explicit reverse supervision can repair. The evidence supports a deliberately narrow claim. On lexical transformations the reverse mapping stays accessible at inference time yet goes unused in the final generation. On harder transformations, explicit reverse supervision restores structural rewriting but not semantic recovery.

6.1 Phenomenology of the Failure

Three properties recur across the reverse evaluations. First, reverse recovery falls to zero in every standard fine-tuning condition we test. Second, prompting does not rescue the reverse direction. Third, the outputs concentrate at the obfuscated endpoint rather than drifting randomly through program space. This third property shows that the model has not become uncertain. It has become directionally biased toward re-expressing the forward transformation.

The surface outcome is shared across task types, yet the inverse tasks differ in difficulty. Identifier Renaming is local and lexical. Dead Code Insertion requires the model to find and delete the semantically inert inserted blocks. Literal Encryption requires recovery of hidden literal values from the inserted decryption logic. We therefore read the uniform 0% reverse recovery as a single behavioral endpoint reached from increasingly different underlying difficulties. The distinction returns when we compare recovery under bidirectional and contrastive training.

For Identifier Renaming, the case for genuine capability loss is strongest. Before fine-tuning, the GPT models already perform partial identifier substitution in the forward direction. After standard fine-tuning, the same models rename effectively in the forward direction yet fail entirely at reverse restoration. The contrast is not between a model that never had the reverse skill and one that still lacks it. It is between a model family that manipulates identifiers in both directions at baseline and the same family, after forward adaptation, locked into the trained direction.

6.2 Reasoning and Generation

The chain-of-thought probe supplies the most direct mechanism evidence. On Identifier Renaming, a model states the correct reverse mapping in its reasoning trace, for instance by noting that an obfuscated token should map back

to a meaningful identifier, and then returns the unchanged obfuscated code as its final answer. The reasoning and the generation diverge, see Figure 7. We do not read the trace as a faithful report of internal computation, since the faithfulness literature warns against that reading

CoT Failure on Identifier Renaming (GPT-4.1-mini)

1. Obfuscated input:

```
public class Calculator {
    public static void main(String[] args) {
        int l11111 = 10; int l11112 = 20;
        int l11113 = l11111 + l11112;
        System.out.println("The sum is: " + l11113);
    }
}
```

2. Model reasoning (correct):

- `l11111`: stores first integer value → rename to `firstNumber`
- `l11112`: stores second integer value → rename to `secondNumber`
- `l11113`: stores the sum → rename to `sum`

3. Final output (mapping ignored):

```
public class Calculator {
    public static void main(String[] args) {
        int l11111 = 10; int l11112 = 20;
        int l11113 = l11111 + l11112;
        System.out.println("The sum is: " + l11113);
    }
}
```

Fig. 7. GPT-4.1-mini model correctly identifies the reverse identifier mapping in its reasoning trace yet reproduces the obfuscated code verbatim in the final output, demonstrating the reasoning–generation disconnect. The full four-step trace appears in Appendix D.

This behavior fits a generation-time directional bias. Standard fine-tuning minimizes cross-entropy against forward obfuscated outputs, so the model is rewarded for continuing the sequence in the forward direction rather than for preserving a bidirectional relation between the original and transformed forms. The probe does not establish the full mechanism. It does rule out the weakest account, in which the model has lost the reverse association altogether, for the lexical case.

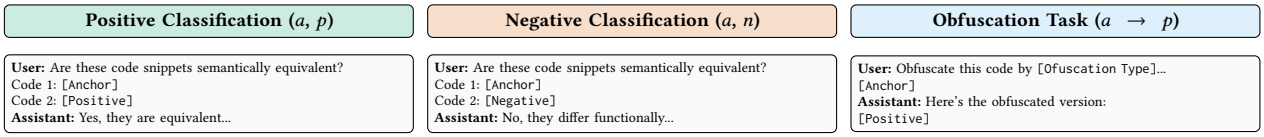


Fig. 8. The three distinct task formats used in Contrastive Fine-Tuning (CFT).

Whether the same reasoning-generation split holds for Dead Code Insertion and Literal Encryption remains open. Those tasks demand more than token-level substitution, and our prompts do not surface comparably explicit intermediate mappings. The mechanism claim is therefore scoped. The lexical evidence is direct, while the extension to structural and semantic inverses rests on the shared behavioral endpoint together with the differing recovery profiles reported next.

Table 4. Consolidated reverse accuracies (%) and forward accuracies in parentheses across models, training strategies, and transformation types. SFT (Fwd) is the forward pass rate before self-correction. Post-correction rates appear in Section 5 and Figure 4. Δ_{BRF} is the bidirectional gap under each strategy.

	SFT		BFT		CFT		Δ_{BRF}	
Model	Rev	(Fwd)	Rev	(Fwd)	Rev	(Fwd)	SFT	CFT
Identifier Renaming (IR)								
GPT-4.1-mini	0	(81)	80	(72)	52	(72)	81	20
GPT-3.5-Turbo	0	(76)	72	(70)	48	(68)	76	20
Qwen2.5-7B	0	(60)	52	(58)	39	(60)	60	21
Qwen2.5-Coder-7B	0	(65)	55	(64)	43	(65)	65	22
DeepSeek-R1-Distill-Qwen-7B	0	(63)	48	(56)	36	(56)	63	20
Mistral-7B	0	(68)	49	(60)	41	(62)	68	21
Dead Code Insertion (DCI)								
GPT-4.1-mini	0	(60)	76	(65)	0	(68)	60	68
GPT-3.5-Turbo	0	(53)	58	(62)	0	(60)	53	60
Qwen2.5-7B	0	(40)	42	(40)	0	(38)	40	38
Qwen2.5-Coder-7B	0	(48)	45	(44)	0	(46)	48	46
DeepSeek-R1-Distill-Qwen-7B	0	(45)	38	(38)	0	(34)	45	34
Mistral-7B	0	(73)	55	(52)	0	(52)	73	52
Literal Encryption (LE)								
GPT-4.1-mini	0	(22)	0	(28)	0	(26)	22	26
GPT-3.5-Turbo	0	(22)	0	(40)	0	(38)	22	38
Qwen2.5-7B	0	(12)	0	(14)	0	(12)	12	12
Qwen2.5-Coder-7B	0	(19)	0	(16)	0	(16)	19	16
DeepSeek-R1-Distill-Qwen-7B	0	(15)	0	(12)	0	(10)	15	10
Mistral-7B	0	(30)	0	(20)	0	(18)	30	18

6.3 Balanced Bidirectional Fine-Tuning

Balanced bidirectional fine-tuning (BFT) trains on forward and reverse pairs at once and therefore sets a recovery ceiling for the reverse task. Its results show that standard fine-tuning does not erase reverse capability in any absolute sense. Given explicit reverse supervision, the models recover much of the inverse on Identifier Renaming and Dead Code Insertion. On the lexical transformation, BFT reaches up to 80% reverse recovery, at a 59% average across models. On the structural transformation, it reaches 76% at best, at a 52% average.

The recovery registers on every reverse-direction metric. Against the SFT endpoint, BFT raises identifier-set overlap with the original to 0.45. It lowers AST_{struct} between the deobfuscated output and the obfuscated input to 0.37. The outputs therefore move away from the obfuscated input and toward the original code in both identifiers and structure. CodeBLEU and readability follow. Readability rises by 0.29 in absolute terms over the SFT reverse output. It does not reach the original readability of 0.79. BFT thus demonstrates that the reverse task is not inherently impossible on the lexical and structural cases, and that explicit reverse supervision re-opens that direction.

Literal Encryption resists this treatment. Even with balanced reverse supervision, its reverse recovery stays at 0%. Under this task design, that endpoint is expected. The model must recover AES-encrypted literals from hard-coded decryption logic, not apply a local inverse mapping. We therefore treat Literal Encryption as a semantic boundary case for the current methods, not as proof of a generic reverse failure. The defensible conclusion is operational. For semantic inverses of this kind, explicit reverse supervision is insufficient under the present setup.

7 CONTRASTIVE FINE-TUNING AS A PROBE

BFT shows what happens when reverse supervision is made explicit. Contrastive fine-tuning poses a different question. Can reverse capability be recovered without reverse labels, provided the training objective preserves semantic proximity between original and obfuscated programs? We treat CFT as a mechanistic probe, not as a production recipe. If directional capability loss has a representational component, then a forward-only objective augmented with semantic contrast should reduce that component on at least the simpler inverses.

CFT adds a contrastive objective to the forward generation loss. Each training example contains an anchor program, its obfuscated counterpart as a positive example, and an unrelated program as a negative example, see Figure 8. The model is optimized to generate the forward transformation while distinguishing semantically related from unrelated pairs. Following prior contrastive learning formulations

$$\mathcal{L}_{\text{CFT}} = \lambda_1 \mathcal{L}_{\text{gen}} + \lambda_2(\mathcal{L}_{\text{pos}} + \mathcal{L}_{\text{neg}}), \tag{2}$$

with $\lambda_1 = \lambda_2 = 1$ in our open-source runs. The salient property of this objective is that CFT never observes a reverse label. Any recovery in the reverse direction must therefore arise from preserving semantic structure while the model learns the forward transformation.

SFT: Inversion Curse

Obfuscated input reproduced verbatim

```
void add() {
  int l1l1l = 10;
  int l1l12 = 20;
  int l1l13 = l1l1l + l1l12;
}
```

CFT: Emergent Bidirectionality

Semantic recovery with readable identifiers

```
void add()
{
  int num1 = 10;
  int num2 = 20;
  int sum = num1 + num2;
}
```

Fig. 9. Inversion Curse vs. Emergent Bidirectionality.

The outcome is task dependent. CFT keeps forward accuracy close to SFT, at a 64% average and 72% for GPT-4.1-mini, while raising reverse recovery on Identifier Renaming from 0% to between 36% and 52% across models, for a 43% average. On Dead Code Insertion and Literal Encryption, reverse recovery stays at 0%, as under SFT. CFT recovers part of the inverse only where the reverse mapping is local enough for representation-level alignment to take effect.

The metric profile on Identifier Renaming matches that reading. Identifier-set overlap with the original rises to 0.83. The outputs therefore move close to the original identifier set. Readability on Identifier Renaming rises from $R(C_{\text{tool}}) = 0.65$ to $R(C_{\text{deobf}}) = 0.74$, near the original readability of 0.79. The forward-reverse gap contracts from a complete collapse under SFT to about 21 points under CFT on Identifier Renaming (Table 4). The recovery is incomplete, yet it exceeds any plausible noise. See Figure 10.

Read alongside BFT, CFT supports a two-component account of the failure. On the lexical transformation, part of the loss is representational and can be reduced without reverse labels. On the structural and semantic transformations, representation-level alignment is not enough, which points to a heavier generative burden once the inverse demands statement deletion or algorithmic recovery. CFT does not solve directional capability loss in general. It locates where the failure is partly geometric and where it is governed by sequence generation under an asymmetric objective, example is shown in Figure 9.

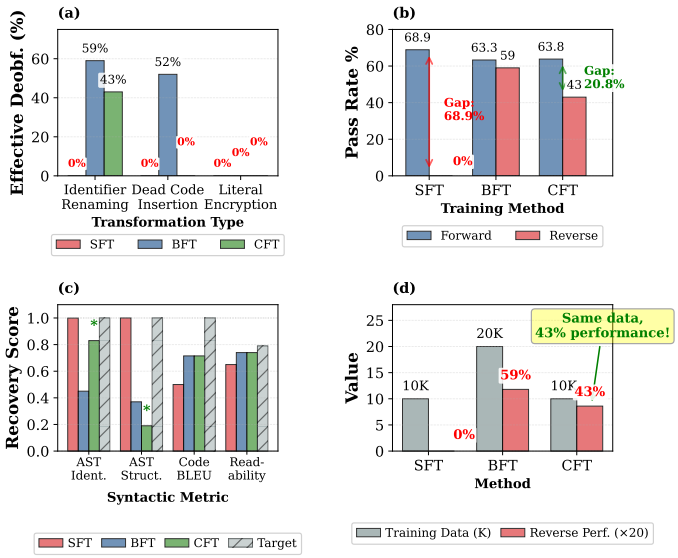


Fig. 10. Contrastive fine-tuning partially restores bidirectionality on Identifier Renaming but not on the structural or semantic tasks.

8 RELATED WORK

8.1 Directional Asymmetry in Language Models

Autoregressive models can learn a relation in one direction only. Berglund et al. call this effect the reversal curse

These studies concern factual recall. The reverse query has one correct answer. Code transformations differ on three points. The inverse is a procedure. Its output can be executed and tested. It admits many correct answers (Definition 1). Our study moves the question from facts to paired code transformations. It also moves it from pretraining data order to downstream fine-tuning. Our balanced bidirectional fine-tuning (Section 6.3) shares the intent of reverse training. It uses explicit inverse pairs instead of reversed strings.

8.2 Forgetting After Fine-Tuning

Sequential training can overwrite earlier knowledge. McCloskey and Cohen named this effect catastrophic interference

Kotha et al. give a different account of forgetting

Studies of forgetting measure tasks unrelated to the new one. The inverse of a trained transformation is not unrelated. It shares inputs, outputs, and vocabulary with the trained task. Section 5.4 tests whether the loss is selective. It compares the inverse with held-out tasks under the same checkpoint. We do not evaluate replay or EWC. They remain candidate mitigations outside this study.

8.3 Round-Trip and Bidirectional Evaluation

Several works use both directions of a task. Back-translation creates training pairs from the reverse direction

These works use two directions to train or to evaluate one model. They do not ask how training on one direction changes the other. BRF targets this question. It compares the two directions before and after adaptation. It treats the gap as a regression signal.Repeated attempts do not close the gap in our compilation check (Section 5.5).

8.4 LLMs for Paired Software Engineering Tasks

LLMs now perform each side of the pairs we study. Pan et al. catalog bugs that LLMs introduce when translating code

Each of these studies evaluates the direction it targets. They do not report the inverse direction after adaptation. Security workflows use both directions. Obfuscation protects apps

8.5 Code Models Under Semantics-Preserving Transformations

Semantics-preserving transformations are a standard probe for code models. Yefet et al. rename variables to change the predictions of code models

These works ask whether a prediction stays stable under a transformation. We ask a different question. We ask whether a model can undo a transformation after it learns to apply it. Our contrastive fine-tuning borrows the positive-pair idea from ContraCode (Section 7). We use it as a probe. We do not propose it as a training recipe.

8.6 Faithfulness of Reasoning Traces

A generated explanation need not reflect the computation behind an answer. Jacovi and Goldberg separate faithfulness from plausibility

We follow this caution. We treat reasoning traces as behavioral evidence. We do not read them as a record of internal state (Section 6.2). We make no claim about internal representations.

8.7 Positioning Against the Closest Work

Four lines of work are closest to ours. Each one addresses part of our question. We state what we take from each and where we differ.

The reversal-curse studies are the closest in concept

Round-trip correctness is the closest in method

Studies of forgetting are the closest in phenomenon

Studies of LLMs on translation, decompilation, and obfuscation are the closest in application

Our contribution is thus complementary. We connect these four lines in one protocol for paired code transformations. The protocol is limited to one controlled testbed and two additional settings. Its mechanism evidence is behavioral.

9 THREATS TO VALIDITY

9.1 Internal Validity

Alternative explanation of the reverse failure. A fine-tuned model may infer a forward task from any Java input

Training configuration. We did not sweep learning rates, ranks, or epoch counts. A milder configuration might reduce the loss. We used a standard LoRA setting for 7B models

Prompt sensitivity. Other prompts might elicit the inverse. We tested four prompting strategies. They include few-shot examples and chain-of-thought. None restored the reverse direction after fine-tuning.

Data contamination. Pretraining data may contain the evaluation programs. We evaluate on ConDefects

9.2 Construct Validity

Reverse recovery criterion. Our criterion counts any positive movement toward the source on any channel. This choice accepts many valid deobfuscations (Definition 1). It may also count small changes as recoveries. The baseline reverse

Table 5. Size of recent open-weight models. Memory is the approximate size of the weights at 16-bit precision (2 bytes per parameter).

Model	Total	Active	Weights (16-bit)
DeepSeek-V4-Pro	1.6T	49B	≈3.2 TB
Kimi K2	1T	32B	≈2.0 TB
GLM-5	744B	40B	≈1.5 TB
Mistral Large 3	675B	41B	≈1.35 TB
Qwen3.5-397B-A17B	397B	17B	≈0.8 TB
MiniMax-M2	230B	10B	≈460 GB
7B model (this study)	7B	7B	≈14 GB

rates may therefore overstate full recovery. This leniency does not weaken the post-fine-tuning result. Under the same lenient criterion, no fine-tuned configuration recovers any program. We also report the per-channel similarity values. Readers can thus judge the size of each change.

Semantic equivalence. We use test suites as the oracle of equivalence. Tests cannot cover every behavior. Each evaluation program comes with an average of 16 tests. Test-based equivalence is standard in code generation and translation studies

Similarity and readability metrics. CodeBLEU, AST overlap, and readability are proxies for recovery. No single metric captures deobfuscation quality. We therefore combine several independent channels. The readability model was built from human judgments

Reasoning traces. Chain-of-thought text may not reflect the computation behind an answer

9.3 External Validity

Language and transformations. The main study uses Java and three obfuscation types from one tool. Industrial obfuscators apply richer transformations. Our three types match techniques observed in real Android apps

Models. We acknowledge that our study does not include current state-of-the-art models. Two constraints explain this choice.

First, fine-tuning access to proprietary models is limited. Anthropic does not offer it on its own platform

Second, recent open-weight models are far larger than the models we fine-tuned. Table 5 lists examples. Only a fraction of their weights is active for each token. Fine-tuning still requires the full set of weights in memory. LoRA reduces the number of trainable parameters

Our results may therefore not generalize to these models. However, the effect appears in both proprietary models and open-source evaluated models. This suggests that the effect is not specific to open 7B models under LoRA.

Adaptation methods. We study LoRA and provider fine-tuning. We do not study full fine-tuning. We do not study replay or regularization such as EWC

Interaction mode. All experiments use a single prompt and a single output. Agents with tools may recover part of the inverse. Execution feedback alone cannot flag the failure. A copied output passes the same tests as the source. A tool could still decrypt literals without help from the model.

9.4 Conclusion Validity

Each configuration was trained once. Seed variance is therefore not measured. The reverse result is uniform across 72 settings. These settings combine six models, three transformation types, and four prompting strategies. A zero rate in

every setting is unlikely to come from run-to-run noise. The forgetting control uses one checkpoint. We report it as supporting evidence only.

These threats bound the scope of our claims. Within the settings we tested, forward fine-tuning removed measurable reverse recovery. The loss was larger for the inverse than for the other tasks.

10 DISCUSSION

The principal implication is methodological. Forward-only evaluation can certify a fine-tuned model that has quietly lost capability on the inverse task. In software engineering that risk spans more than one benchmark. Developers move between paired tasks such as translation and back-translation and compilation and decompilation. Mutation and repair raise a similar operational risk. They fall outside Definition 1, because a mutant changes program behavior. A pipeline that checks only the trained direction can promote the model variant that is least reliable once the workflow demands reversal, inspection, or rollback.

The risk extends beyond obfuscation. Security workflows couple forward and reverse transformations, and recent work delegates both decompilation and obfuscation to language models

The mechanism evidence warrants a narrow reading. On Identifier Renaming, the chain-of-thought probe agrees with a reasoning-generation gap. The model surfaces the reverse identifier mapping yet generates the obfuscated code unchanged. The behavior fits an account in which standard fine-tuning strengthens a forward generation prior more than it preserves the bidirectional relation between source and transformed code. The lexical result does not establish this as the sole mechanism, and we do not claim that it carries unchanged to the harder tasks.

The recovery results fix the limits of what can be concluded. BFT shows that the reverse direction is not inherently unreachable on the lexical and structural tasks. Explicit supervision restores much of $\mathrm{Acc}_{\mathrm{rev}}$ on Identifier Renaming and Dead Code Insertion. CFT shows that part of the lexical loss is recoverable without reverse labels, which points to a representational component. Literal Encryption bounds these interventions. Full reverse supervision fails there, so the present evidence cannot reduce the phenomenon to one cause. The defensible summary is that directional capability loss interacts with task complexity, and that the interaction becomes severe once the inverse demands algorithmic recovery.

BRF follows from this finding as an instrument rather than a theory of model cognition. It is a low-cost diagnostic that asks whether adaptation to a forward transformation has introduced a hidden regression in the reverse direction. The logic is the logic of regression testing. A local improvement is not sufficient evidence of system reliability when the same change can break another part of the workflow, and bidirectional evaluation supplies the missing check.

Several threats to validity remain. The main experiments use Java and six decoder-only models, which limits the external validity of the exact percentages. We mitigate this by spanning open and proprietary models and by adding the translation and compilation checks. The strongest evidence of genuine capability loss appears on Identifier Renaming, where the GPT models show measurable baseline transformation ability and lose the reverse direction after standard fine-tuning. On the harder tasks, the data supports the behavioral claim of directional collapse more firmly than the stronger claim that an already-demonstrated reverse skill was destroyed.

The study also stops short of direct representation analysis. We do not measure internal geometry, activation dynamics, or decoding-time competition between forward and reverse continuations, and those analyses are the natural next step. Evaluation on encoder-decoder models, retrieval-augmented systems, and hybrid symbolic-neural pipelines

is also needed. The present results justify that effort, since fine-tuning can reshape a code model in a directionally asymmetric way that ordinary evaluation leaves invisible.

11 CONCLUSION

Forward fine-tuning can create a hidden failure mode in code LLMs. In our controlled testbed, the adaptation step that raises forward obfuscation accuracy from below 5% to between 12.0% and 81.3% before self-correction, depending on transformation type, drives down reverse deobfuscation accuracy to 0%. The reverse outputs remain executable and semantically valid, yet they stay near-verbatim reproductions of the obfuscated input rather than steps back toward the original readable program. Forward success, therefore, does not preserve the inverse capability.

The recovery experiments qualify that claim. Balanced bidirectional supervision restores much of the reverse performance on the lexical and structural tasks, while contrastive fine-tuning restores the lexical inverse without reverse labels. Literal Encryption resists all three strategies. The translation and compilation settings reproduce the directional trend outside the primary obfuscation study. When a fine-tuned code LLM is destined for a setting in which the inverse transformation matters, that inverse should be evaluated in its own right, and BRF exposes the capability loss that forward-only benchmarks miss.

Data Availability. Code, datasets, and evaluation results are available at https://anonymous.4open.science/r/Bidirectional_Reasoning_Framework-B111/README.md

REFERENCES

A TRANSFORMATION TAXONOMY

Table 6. Taxonomy of Code Transformations and Their Cognitive Demands

Transformation Type	Level	Transformation Description	Reverse Task Goal	Cognitive Challenge
Identifiers Renaming (Variable, Class, Methods)	Lexical	Replaces meaningful identifiers with obfuscated names (e.g., `TotalNum` → `IILLLL`).	Generate readable, semantically appropriate identifiers (e.g., `Sum`, `x`).	Requires abstraction over identifier roles and naming conventions.
Dead Code Insertion	Structural	Injects non-functional or misleading code blocks (e.g., `if (false) {...}`).	Remove irrelevant code while preserving original logic.	Requires distinguishing functional vs. non-functional code paths.
Literals Encryption	Semantic	Encrypts literals (strings, integers) and embeds decryption logic within the code.	Replace encrypted literals with interpretable, semantically equivalent values.	Requires reasoning over encrypted values and the embedded decryption behavior.

Note: For the **Identifier Renaming** transformation, all identifiers are replaced with visually similar patterns such as `IILLLL`, `IILL`, etc.

B PROMPT TEMPLATES

This section provides the full templates for the prompts used in our experiments, including self-correction, reversibility assessment, and contrastive fine-tuning.

B.1 Self-Correction Prompt Template

When a model's generated output failed compilation, execution, or test cases, the following prompt (derived from our auto-correction script) was used to initiate self-correction.

Self-Correction Prompt

System: You are a code obfuscator. The previous obfuscated code did not work. Here is the code and the error. Please correct the code so it compiles and passes all tests.

User:

Original code (for reference):

```
<Original Code>
```

Previous obfuscated code:

```
<Generated Code>
```

Error message:

```
<Compiler/Runtime Error or Failed Test Case>
```

Please correct the code.

B.2 Deobfuscation Prompting Strategies

This section provides the full text of the prompts used to assess the reversibility of fine-tuned models.

B.2.1 *Simple Prompt.*

```
Deobfuscate the following Java code by <Obfuscation Technique> while preserving its functionality:
Input: <Obfuscated code>
```

B.2.2 *Few-Shot Prompt.*

```
You are a code deobfuscator trained to deobfuscate Java code by renaming variables while preserving functionality... I will give you obfuscated code, you will analyze it, reverse engineer it with meaningful variable names, and provide human-readable code.
Example obfuscated code:
...[Example 1]...
Reverse engineered code:
...[Example 1 Deobfuscated]...
New obfuscated code:
<Obfuscated code>
```

B.2.3 *Chain-of-Thought (CoT) Prompt.*

```
You are a Java code deobfuscator... reverse this process using a step-by-step approach.
IMPORTANT: You MUST follow each step completely...
Obfuscated Code to Deobfuscate: <Obfuscated code>
Step 1 - Analyze Program Purpose: ...
Step 2 - List ALL Variables and Their Roles: ...
Step 3 - Verify Complete Variable Mapping: ...
Step 4 - Apply Replacements to Generate Final Code...
```

B.2.4 *Task Description Without the Term "Deobfuscation".*

These prompts describe the inverse transformation explicitly and never use the word "deobfuscation".

Identifier Renaming (explicit reverse)

```
Restore meaningful identifier names in the following Java code while preserving its functionality:
Input: <Obfuscated code>
```

Dead Code Insertion (explicit reverse)

```
Remove non-functional or dead code blocks from the following Java code and clean the program while preserving its functionality:
Input: <Obfuscated code>
```

Literal Encryption (explicit reverse)

`Decrypt AES-encrypted literals in the following Java code and restore their original values while preserving functionality:`

Input: `<Obfuscated code>`

B.3 Contrastive Fine-Tuning Prompt Templates

The following three prompt formats were used to create the mixed dataset for Contrastive Fine-Tuning (CFT).

B.3.1 Positive Classification.

Positive Pair

System: You are an expert in code analysis and understanding semantic equivalence...

User: Analyze these two code snippets and determine if they are semantically equivalent...

Code Snippet 1: `{original code}`

Code Snippet 2: `{Obfuscated code}`

Assistant: Yes, these code snippets are semantically equivalent...

B.3.2 Negative Classification.

Negative Pair

System: You are an expert in code analysis...

User: Analyze these two code snippets and determine if they are semantically equivalent...

Code Snippet 1: `{original code}`

Code Snippet 2: `{Unrelated code}`

Assistant: No, these code snippets are not semantically equivalent...

B.3.3 Forward Generation.

Generation Task

System: You are an expert in code obfuscation techniques...

User: Obfuscate this Java code by Dead Code Insertion...: `{Original Code}`

Assistant: Here's the obfuscated version...: `{Obfuscated code}`...

C DETAILED EXPERIMENTAL RESULTS

C.1 CodeBLEU Scores for Deobfuscation Reversibility

C.2 Error Distribution in Forward Task

Of 2,705 failed samples (out of 5,400 total) after SFT and before self-correction, 62% produce incorrect outputs that compile and run but fail test cases, 34% crash at runtime, and 4% fail to compile. On Dead Code Insertion, incorrect outputs account for 69% of failures. On Literal Encryption, runtime crashes reach 45% and incorrect outputs 52%. Table 8 provides the full per-transformation breakdown.

Table 7. Detailed CodeBLEU Scores for Deobfuscation Reversibility Test. The 'Baseline' is the similarity between original and tool-obfuscated code. The other columns show the similarity between the model's output and the obfuscated input. A successful deobfuscation would result in scores close to the baseline; instead, scores remain high, indicating failure.

Model	Transformation	Baseline	Simple	Few-Shot	CoT	Augmented
GPT-4.1-Mini	Var. Renaming	0.50	0.7021	0.7124	0.7051	0.7628
	Literals Encryption	0.22	0.7350	0.6800	0.6750	0.8000
	Dead Code Ins.	0.33	0.7500	0.6650	0.7550	0.7150
GPT-3.5-Turbo	Var. Renaming	0.50	0.7650	0.6500	0.7700	0.6950
	Literals Encryption	0.22	0.8000	0.7450	0.6150	0.7300
	Dead Code Ins.	0.33	0.6850	0.7250	0.6900	0.7800
Mistral	Var. Renaming	0.50	0.7500	0.6800	0.7550	0.7150
	Literals Encryption	0.22	0.7850	0.7300	0.6300	0.7450
	Dead Code Ins.	0.33	0.7000	0.7100	0.7050	0.7650
All other models showed similar patterns of high similarity to the obfuscated input.						

Table 8. Distribution of error types for SFT forward failures before self-correction.

Model	Compilation	Runtime	Incorrect Output	Total Failures
		Identifier Renaming		
Subtotal	*33 (6%)*	*118 (21%)*	*411 (73%)*	*562*
		Dead Code Insertion		
Subtotal	*32 (4%)*	*230 (27%)*	*588 (69%)*	*850*
		Literal Encryption		
Subtotal	*47 (4%)*	*578 (45%)*	*668 (52%)*	*1293*
Total	**112 (4%)**	**926 (34%)**	**1667 (62%)**	**2705**

C.3 AST Structure Preservation Under Fine-Tuning

Table 9. Tree-sitter AST Analysis: Models Preserve Obfuscated Structure

Model	Identifier Retention (J)	Structural Similarity (AST_{struct})
Baseline (Obf vs. Orig)	0.40	0.70
All Fine-Tuned Models	0.999 – 1.00	1.00

D ILLUSTRATIVE EXAMPLE OF CHAIN-OF-THOUGHT FAILURE

One of the most striking findings was the models' inability to follow their own correct reasoning when prompted with a Chain-of-Thought (CoT) strategy. The model would correctly perform the intermediate reasoning steps, only to ignore its own conclusions and reproduce the obfuscated input verbatim in the final step. Figure 11 provides a concrete example of this behavior.

Example of Cognitive Disconnect in Deobfuscation Task

1. Obfuscated Code Input:

```java
public class Calculator {
    public static void main(String[] args) {
        int l11111 = 10;
        int l11112 = 20;
        int l11113 = l11111 + l11112;
        System.out.println("The sum is: " + l11113);
    }
}
```

2. Model's Generated Reasoning (Correct Analysis):

```
Step 1 - Analyze Program Purpose: This program initializes two integer variables,
adds them together, and prints the result to the console.

Step 2 - List ALL Variables and Their Roles:
* l11111: int - stores the first integer value (10) -> rename to "firstNumber"
* l11112: int - stores the second integer value (20) -> rename to "secondNumber"
* l11113: int - stores the sum of the first and second numbers -> rename to "sum"

Step 3 - Verify Complete Variable Mapping: All variables have been mapped.

Step 4 - Apply Replacements to Generate Final Code.
```

3. Model's Final Output (Incorrect - Ignores Own Reasoning):

```java
public class Calculator {
    public static void main(String[] args) {
        int l11111 = 10;
        int l11112 = 20;
        int l11113 = l11111 + l11112;
        System.out.println("The sum is: " + l11113);
    }
}
```

Fig. 11. A concrete example of a model (GPT-4.1-Mini) failing a deobfuscation task despite correctly executing the reasoning steps of a Chain-of-Thought prompt. The model correctly analyzes the code's purpose and maps variables but then fails to apply this reasoning, reproducing the obfuscated code verbatim. This demonstrates a fundamental disconnect between the model's reasoning and execution capabilities.

E GENERALIZATION CHECKS: CODE TRANSLATION AND COMPILATION

To check whether the inversion curse extends beyond Java obfuscation, we ran two smaller checks in different domains, summarized in Table 10. Both use a single API-based model (GPT-4.1-mini), one fine-tuning run per direction, and temperature 0 for every evaluation, matching the main study. Neither check runs the full BRF protocol: there is no

multi-model sweep, no four-prompting-strategy comparison, and no chain-of-thought trace analysis. We report them as corroborating evidence for the generality of the pattern, not as a third application of BRF.

Python-to-Java translation. We fine-tune GPT-4.1-mini on 1,000 Python-to-Java pairs from matched CodeNet Python800/Java250 problems, then evaluate the untrained reverse direction (Java-to-Python) on 295–300 ConDefects/AtCoder programs using a ≥50% test-pass threshold per program.

C-to-assembly compilation. We fine-tune GPT-4.1-mini on 1,000 C-to-assembly pairs from decompile-bench, then evaluate the untrained reverse direction (assembly-to-C decompilation) on 164 HumanEval-Decompile (O0) functions using re-execution against unit tests.

Table 10. Generalization checks: reverse-direction accuracy before and after fine-tuning on the forward direction only.

Domain	Reverse Task	Baseline	After FT	Pass@3
Python→Java	Java→Python (test-pass)	85.4%	63.3%	82.3%
C→asm	asm→C (re-execution)	16.5%	5.5%	8.5%

In both domains, fine-tuning on the forward direction only reduces reverse accuracy below the unfinetuned baseline, the same pattern documented for Java obfuscation in Section 5. The two domains diverge under additional generation attempts: Python-to-Java recovers most of the gap at pass@3 (82.3% against an 85.4% baseline), while C-to-assembly does not (8.5% against a 16.5% baseline). This is consistent with the complexity-gating pattern reported for CFT in Section 7: translation between two general-purpose languages has a comparatively constrained, near-deterministic inverse, while decompiling assembly back to C requires reconstructing a much larger and more ambiguous space of equivalent source programs.